SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL - 2

SHORT ANSWER TEST

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Short Answer Test**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO1 Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	**B Reddy Balu**	**Reg. No.**	**192312499**
Section/Batch	**Slot A**	**Date**	**21-10-2026**

Code of Conduct:

I, ***B Reddy Balu (192312499)*** , certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Balu

Question 1: A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer

Testing as an engineering activity means that testing is planned, designed, executed, measured and improved in a disciplined, repeatable way, just like any other engineering work. Random (ad hoc) testing has no defined objectives, no coverage measure and no repeatability, so results vary from tester to tester.

Redesigned approach

- **Analyse requirements and risks:** Identify what must be tested and which features are most critical.
- **Plan:** Define test objectives, scope, schedule, resources, entry and exit criteria.
- **Design test cases:** Use techniques such as equivalence partitioning, boundary value analysis, decision tables and state transition testing, with defined inputs and expected results.
- **Execute and log:** Run the designed tests, record actual results and report defects with clear steps to reproduce.
- **Measure and improve:** Track coverage, defects found and defect trends, then refine the tests.

How structured test design improves quality

- **Coverage:** Every requirement is mapped to at least one test (traceability), so fewer features are missed.
- **Repeatability:** Anyone can re-run the same tests and get comparable results.
- **Defect detection:** Techniques target the places where defects usually hide (boundaries, invalid inputs, state changes).
- **Efficiency:** Redundant tests are removed and effort is focused on high-risk areas.

Example: For a login page, random testing may try only a valid and one invalid password. Structured design adds cases for blank fields, minimum and maximum length, special characters, SQL injection, wrong password three times (account lock) and session timeout, so far more defects are found.

Question 2: A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer

A defined testing process is a documented set of activities, roles, entry/exit criteria and outputs that every project follows. For a banking system, where an error means financial loss and regulatory trouble, this is essential.

Phases of a well-defined testing process

- **Test planning and control:** Scope, strategy, risks, resources, schedule.
- **Test analysis and design:** Derive test conditions and test cases from requirements.
- **Test implementation and execution:** Prepare test data and environment, run tests, log results.

- **Evaluating exit criteria and reporting:** Check coverage and defect status before release.
- **Test closure:** Archive testware, record lessons learned.

Benefits with examples

- **Fewer production failures:** Planned tests for fund transfer, balance update and interest calculation catch errors such as a debit without a matching credit.
- **Handling real-time risks:** Defined concurrency and performance tests reveal problems like two simultaneous withdrawals from the same account.
- **Security and compliance:** Mandatory security and audit-trail tests meet regulatory rules.
- **Consistency and traceability:** Every requirement is linked to tests, so nothing is skipped.
- **Measurable quality:** Metrics such as defect density and defect removal efficiency show readiness for release.
- **Reuse:** A maintained regression suite protects existing features when new services (for example UPI) are added.

Question 3: During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer

Testing should run in parallel with every SDLC phase, not only at the end. The V-model shows this clearly: each development phase has a matching test level.

Testing aligned to SDLC phases (V-model)

- **Requirements:** Review requirements; prepare the acceptance test plan.
- **System design:** Prepare the system test plan (end-to-end order flow).
- **Architecture / high-level design:** Prepare integration tests (cart, payment gateway, inventory, shipping).
- **Module / low-level design:** Prepare unit tests (cart total, discount and tax calculation).
- **Coding:** Perform unit testing, static analysis and code reviews.
- **After coding:** Run integration testing, system testing, then user acceptance testing (UAT).
- **Maintenance:** Regression testing after each change or release.

How process discipline improves quality

- **Entry and exit criteria:** A phase starts or ends only when defined conditions are met (for example no open critical defects before system testing).
- **Early defect detection:** Defects are caught at the level where they are cheapest to fix.
- **Traceability:** Requirements map to test cases, giving full coverage.
- **Consistency and metrics:** Standard templates and metrics let management control quality.

Result: integration defects (for example the payment gateway returning success but the order not being created) are caught at integration level instead of reaching customers.

Question 4: A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer

Definitions

- **Error (mistake):** A human action that produces an incorrect result, for example a developer misreading the requirement.
- **Defect (fault / bug):** A flaw in the code or document caused by an error, which can cause the software to behave incorrectly.
- **Failure:** The deviation of actual behaviour from expected behaviour when the software runs; this is what users and testers observe.

Classification of the given issue

A mismatch between expected and actual output seen during validation is a FAILURE. It is the visible symptom. Behind it there is a DEFECT in the code, and behind the defect there was an ERROR made by a person.

Relationship

Error leads to Defect, and Defect (when executed under the right conditions) leads to Failure. Not every defect produces a failure, because the faulty code may never be executed or the conditions may never occur.

Example: Requirement: a password must be at least 8 characters. The developer misreads it (error) and writes the condition length > 8 (defect). A tester enters an 8-character password and it is rejected (failure: expected 'login successful', actual 'invalid password'). The fix is made at the defect, and the cause is prevented by reviewing requirements.

Question 5: A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer

Principle applied: Early Testing (Shift-Left)

Testing activities should start as early as possible in the SDLC and focus on defined objectives. The cost of fixing a defect rises sharply with each phase it survives: a defect that costs one unit to fix in requirements may cost many times more in testing and far more in production.

How to apply it

- Review and inspect requirements and design documents (static testing).
- Prepare test cases while requirements and design are being written.
- Perform unit testing and code reviews during development, with continuous integration.
- Involve testers in planning and design meetings.

How it improves effectiveness

- Defects are found where they are introduced, so they are cheaper and quicker to fix.
- Less rework and fewer schedule delays.
- Ambiguities in requirements are removed before they spread into code.
- Higher defect removal efficiency and better product quality.

Example: In a bank loan application, the requirement says 'high-income applicants get a discount'. A tester reviewing it asks 'what is high income?'. Clarifying it as 'above 10 lakh per year' costs a few minutes. If found after release, it would require code changes, retesting, redeployment and possibly refunds.

Question 6: During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer

Tester contributions during requirement analysis

- **Requirement review:** Check requirements for ambiguity, incompleteness, inconsistency and untestable statements.
- **Asking questions:** Raise clarification queries to business analysts and stakeholders.
- **Testability check:** Ensure each requirement is measurable and verifiable, with clear acceptance criteria.
- **Early test planning:** Identify test conditions, risks and draft high-level test cases.
- **Traceability:** Start a requirements traceability matrix (RTM).
- **Participation in walkthroughs and inspections:** Give the user and test viewpoint.

Example: The requirement 'the system shall calculate the correct medicine dosage' is ambiguous. A tester asks: based on weight or age? what units? what maximum limit? what happens for a missing weight? Once clarified, the requirement is precise and testable.

Importance of early involvement

- Defects found in requirements are the cheapest to fix.
- Prevents ambiguities from passing to design and code, reducing rework.
- Builds a shared understanding between developers, testers and business users.
- Critical in healthcare, where a defect can affect patient safety.
- Gives testers time to prepare better test cases, data and environment.

Question 7: A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer

How defects originate in the lifecycle

- **Requirements phase:** Ambiguous, missing or conflicting requirements. Studies show this is where a large share of defects are injected.
- **Design phase:** Designers interpret unclear requirements in different ways, producing wrong logic and wrong interfaces.
- **Coding phase:** Developers make assumptions to fill gaps, or make logic and syntax mistakes.
- **Testing phase:** Test cases derived from unclear requirements are themselves wrong, so defects are missed.
- **Deployment and maintenance:** Wrong configuration or changes without impact analysis add new defects.

Example: The requirement says 'pay overtime at a higher rate'. It does not state the rate, the threshold or whether holidays count. The designer assumes 1.25x after 48 hours, the developer codes 1.5x after 40 hours, and the tester has no precise expected value. The result is incorrect salaries.

How requirement clarity prevents such issues

- Write requirements that are unambiguous, complete, consistent, measurable and testable (for example, an SRS following IEEE 830).
- Use examples, formulas, use cases and acceptance criteria (overtime = 1.5 x hourly rate for hours above 40 per week).
- Hold reviews and inspections with testers and stakeholders.
- Maintain a glossary and traceability so each requirement links to design, code and tests.
- Track defect origin with root cause analysis to improve the process.

Question 8: A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer

Classification

This is an input handling / boundary condition defect, a functional (logic) defect in the code. Its severity is high or critical because it results in a system crash. It is also a test coverage gap, since the test design did not include boundary values.

Justification

- The failure occurs only for values at the edge of valid ranges, which points to an incorrect comparison (for example < instead of <=) or an off-by-one error, or to missing validation.
- Normal values work correctly, so the basic logic is fine; only the boundary handling is wrong.
- A crash is a major-severity failure type, and the team suspects poor coverage, so test design is also at fault.

How boundary value analysis (BVA) would have prevented it

BVA is a black-box technique that tests values at and around the edges of each input range, because defects cluster at boundaries. For a range min to max, test: min-1, min, min+1, a nominal value, max-1, max, max+1.

Example: An age field accepts 18 to 60. BVA tests 17, 18, 19, 40, 59, 60 and 61. If the code uses age < 60 instead of age <=

60, the test at 60 exposes the defect during testing, before users see a crash. BVA is usually combined with equivalence partitioning for best coverage.

Question 9: A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer

A defect repository is a central database of all defects, with details such as ID, description, module, severity, priority, status, phase of origin, root cause and fix. It becomes a knowledge base when it is analysed.

How it supports better test case design

- **Identify defect-prone modules:** Analyse defect counts per module and focus more tests there.
- **Learn defect patterns:** Repeated types (date handling, null values, boundaries) lead to specific new test cases.
- **Build regression tests:** Every fixed defect gets a test case so it cannot return unnoticed.
- **Create checklists:** Defect categories become review and test checklists.
- **Improve test data:** Real failing inputs from past defects become test data.

Example: The repository shows 12 defects related to date formats. The team adds test cases for leap years, month-end dates, different regional formats and invalid dates.

Role in improving software quality

- Prevents repeated defects across releases.
- Supports root cause analysis and process improvement.
- Provides metrics (defect density, defect removal efficiency, trends) for decision making.
- Improves communication and traceability between testers and developers.

Question 10: An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer

Identification as a functional defect

A functional defect occurs when the system does not behave as specified by its functional requirements. Here the withdrawal feature does not implement the transaction rules correctly (check balance, check limit, debit account, dispense cash, update the record, issue receipt).

Examples: the account is debited but no cash is dispensed; withdrawal is allowed when the amount exceeds the balance; an amount exactly equal to the balance is rejected; the balance is not rolled back after a hardware failure.

Classification: functional defect, severity critical (financial impact), found in the transaction logic module.

How it could have been prevented

- **Requirement and design reviews:** Clear business rules for withdrawal and failure handling.
- **Decision table testing:** Cover combinations of balance, limit, card status and cash availability.
- **State transition testing:** Cover the transaction flow and error states.
- **Boundary value analysis:** Test amount equal to balance, daily limit, minimum amount.
- **Negative and failure tests:** Network failure, cash jam, power loss, with rollback verification.
- **Unit and integration testing:** Test the transaction module and its link to the bank server.
- **Code review and regression testing:** Catch logic mistakes early and protect existing behaviour.

Question 11: A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer

Defect clustering

Defect clustering is a testing principle that states a small number of modules contain most of the defects (the Pareto principle, roughly 80% of defects in 20% of modules). Defects tend to cluster in complex, frequently changed or poorly understood areas.

Identifying testing priorities

- Analyse defect data per module and calculate defect density.
- Rank modules by defect count, severity and business criticality.
- Consider risk factors: high complexity, frequent changes, new or inexperienced developers, many interfaces.
- Give top-ranked modules (for example payment and checkout) the most test effort, deeper techniques, more regression and extra code reviews.
- Give stable, low-defect modules lighter, risk-based testing.

How it improves efficiency

- Effort and time go where the defects are, so more defects are found per hour of testing.
- Better use of limited resources and shorter test cycles.
- Early attention to risky modules reduces the chance of serious defects reaching production.

Caution: do not ignore other modules completely, and refresh test cases regularly (pesticide paradox), because the clusters can move over time.

Question 12: A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer

Classification of the defect

- **By type:** Non-functional, performance defect (response time and scalability under load). Related checks include load, stress and endurance testing.
- **By severity:** Major (or critical if the application becomes unusable at peak time).
- **By priority:** High, because it affects many users and the business.
- **By probable root cause:** Inefficient database queries, missing indexes, memory leaks, no caching, poor connection pool or insufficient server capacity.
- **By phase detected:** Found in performance (system) testing, or in production if missed.

Defect classification techniques include classification by type, severity and priority, by phase of origin, by root cause, and Orthogonal Defect Classification (ODC).

How classification helps resolution

- Routes the defect to the right team (database, backend, infrastructure).
- Sets correct fixing order by severity and priority.
- Supports root cause analysis and trend analysis (for example repeated performance issues in every release).
- Guides targeted tests (load and stress scripts) and tools.
- Provides metrics for management to plan resources and process improvement.

Question 13: A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer

Regression testing confirms that new changes have not broken existing functionality. Historical defect data shows where the product has been weak, so it helps decide what to retest.

Reusing historical defect data

- **Convert fixed defects into test cases:** Every previously fixed defect gets a regression test to make sure it does not reappear.
- **Select tests for defect-prone modules:** Modules with many past defects are included in every regression cycle.
- **Prioritise by severity and frequency:** High-severity and frequently occurring defect areas run first.
- **Impact analysis:** Link defects to code, requirements and tests to find what a new change might affect.
- **Trim the suite:** Remove or reduce tests that have never found defects in many releases.
- **Automate high-value tests:** Automate stable, repeated and high-yield tests.

Improvement in coverage and efficiency

- Better coverage of risky areas and previously failing scenarios.
- Shorter execution time because only relevant and valuable tests are run.
- Early detection of reopened or reintroduced defects.

- More confident release decisions based on data.

Question 14: During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer

Role of the tester in minimizing such defects

- **Clarify requirements and interfaces:** Ask questions and confirm API, data format and module interface specifications with developers.
- **Participate in reviews:** Join requirement, design and code reviews and give the test viewpoint.
- **Share test plans and cases:** Developers see what will be tested and can test the same conditions first.
- **Write clear defect reports:** Include steps to reproduce, expected and actual results, logs, screenshots, severity and environment.
- **Regular communication:** Attend daily stand-ups and defect triage meetings; use a common defect tracking tool.
- **Plan integration testing early:** Prepare the integration strategy, stubs and drivers, and the test environment.
- **Verify fixes and run regression:** Retest quickly and give prompt feedback.

How early involvement helps

- Interface mismatches and misunderstood requirements are found at the design stage, when they are cheap to fix.
- Creates a shared understanding, so the same specification is used by developers and testers.
- Reduces blame and rework and builds trust between teams.
- Leaves more time for good test design and a smoother integration phase.

Question 15: A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer

Defect prevention aims to stop defects from being introduced, rather than only detecting them after the fact. It addresses the root causes of recurring defects.

Defect prevention techniques

- **Root cause analysis:** Use Pareto charts, fishbone diagrams and 5-Whys on repeated defects found in the defect repository.
- **Causal analysis meetings:** Review defect causes after each release and agree on process changes.
- **Reviews and inspections:** Peer reviews and checklists for requirements, design and code.
- **Coding standards and static analysis:** Automatic detection of common coding mistakes.
- **Training:** Train developers and testers on the areas where defects keep appearing.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	